

Implementación y análisis del desempeño de un pipeline para el estudio de **amplicones 16S** en un clúster energéticamente eficiente basado en **Raspberry Pi 5**

Cristian Alberto Solano Torres
Zamir Francisco Granados Peñaloza

Director: Bernardo Andrés Benavides Arévalo, PhD

Escuela de Ingeniería de Sistemas e Informática · UIS · 2026

Contenido

1

Contexto y problema

2

Objetivos y metodología

3

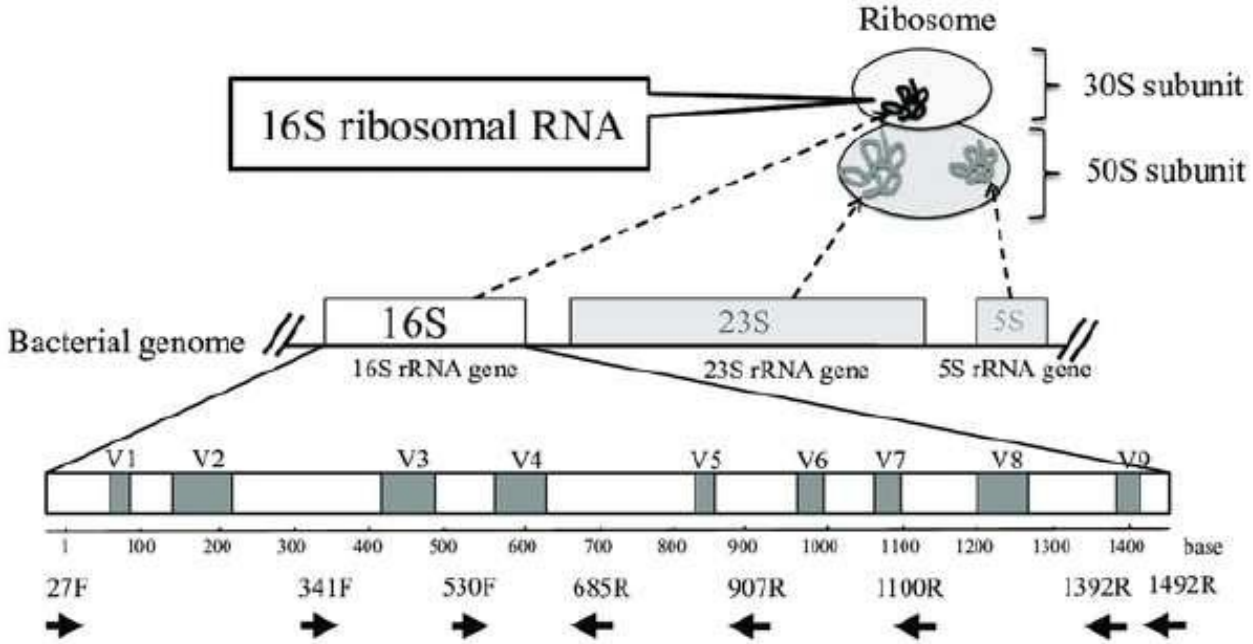
Resultados por fase

4

Conclusiones y trabajo futuro

#LaUISqueQueremos

El gen 16S y su análisis



Estructura del RNA ribosomal 16S (Fukuda y cols., 2016)

El gen

Está en todas las bacterias y arqueas

≈ 1,500 pares de bases (A, C, G, T)

Regiones V1–V9 distinguen géneros, como un código de barras

Su análisis

Secuenciador: convierte el ADN en un archivo con millones de lecturas (reads)

Pipeline: toma ese archivo y dice a qué bacteria pertenece cada lectura

El problema

8 – 100 GB
de RAM para Kraken 2, según la base

300 – 600 W
una estación de trabajo

300 – 5000 W
un nodo HPC

1. Estos análisis se concentran en **estaciones de trabajo y sistemas HPC**
2. GPU y FPGA aceleran, pero son **costosas y difíciles de configurar**
3. **No hay reportes** de la mayoría de pipelines 16S **sobre ARM64**

#LaUISqueQueremos

Pregunta de investigación y propuesta

PREGUNTA DE INVESTIGACIÓN

¿Cuál es la eficiencia energética y el rendimiento de un pipeline de amplicones 16S en un clúster Raspberry Pi 5, frente a una estación x86 estándar (lecturas analizadas por unidad de energía, reads/Wh)?

Propuesta

ARQUITECTURA CLÚSTER

Clúster de 5 nodos

Raspberry Pi 5 (ARM64)

CAPACIDAD POR NODO

4 núcleos Cortex-A76

@ 2.4 GHz y 16 GB RAM

BENCHMARK COMPARATIVO

Dos laptops x86

Intel i5-10300H

MÉTRICA CENTRAL

Reads/Wh

Lecturas por vatio-hora



Raspberry Pi 5

#LaUISqueQueremos

Objetivos

Objetivo general

Analizar el desempeño y la eficiencia energética de un pipeline para amplicones 16S en un clúster de Raspberry Pi 5 frente a una estación x86 estándar.

1

Diseñar

Arquitectura hardware y software del clúster

2

Portar

Pipeline 16S a ARM64, en paralelo

3

Validar

Resultados biológicos frente a x86

4

Evaluar

Tiempo y energía → reads/Wh

#LaUISqueQueremos

Metodología: ciclo DMAIC (Six Sigma)

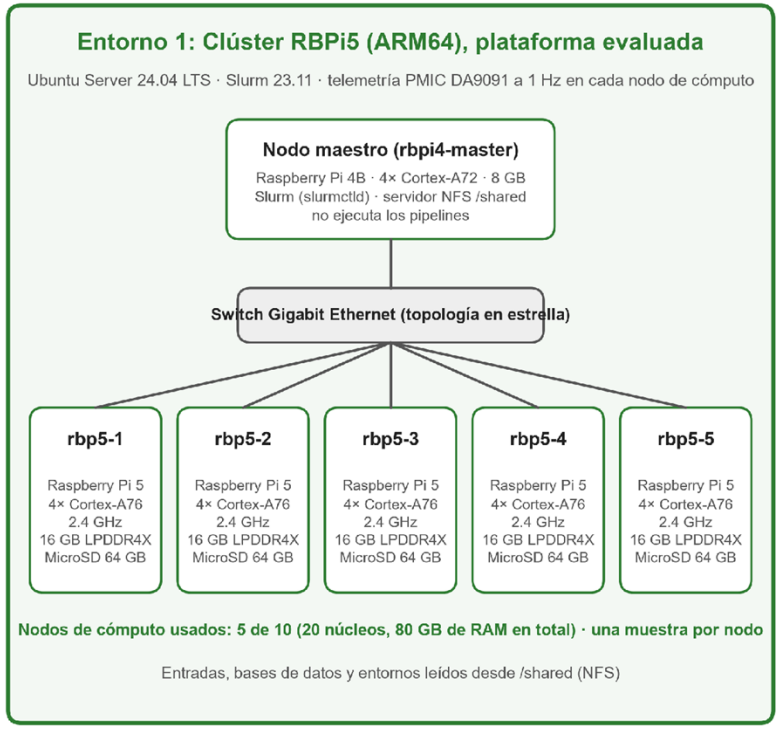
Se aplicó en cada una de las 4 fases

- **D** **Definir:** requisitos y criterios de éxito
- **M** **Medir:** estado inicial y especificaciones
- **A** **Analizar:** restricciones y cuellos de botella
- **M** **Mejorar:** aplicar las soluciones
- **C** **Controlar:** verificar antes de seguir



Ciclo DMAIC

Entornos experimentales



Entorno 2: Laptop x86 con Ubuntu nativo

Referencia x86 principal

Procesador	Intel Core i5-10300H (x86-64) 4 núcleos / 8 hilos, 2.5–4.5 GHz
Sistema	Ubuntu 24.04 LTS (nativo)
Memoria	16 GB DDR4
Disco	SSD NVMe 512 GB
Telemetría	Intel RAPL (sysfs), 1 Hz

Entorno 3: Laptop x86 con WSL2

Referencia complementaria

Procesador	Intel Core i5-10300H (x86-64) 4 núcleos / 8 hilos, 2.5–4.5 GHz
Sistema	Windows 11 + WSL2 (Ubuntu 22.04 LTS)
Memoria	32 GB DDR4 (28 GB asignados a WSL2)
Disco	SSD NVMe 500 GB
Telemetría	Intel RAPL (LibreHardwareMonitor), 1 Hz

Los tres entornos de experimentación

Clúster RBPi5
5 × Raspberry Pi 5 · PMIC: mide la placa completa

x86 nativo
i5-10300H · Ubuntu 24.04 · RAPL: solo el procesador

WSL2
Mismo i5 · Windows 11 + WSL2 · RAPL vía LibreHardwareMonitor

Fase 1: diseño del clúster

5 nodos
Raspberry Pi 5

20 núcleos
Cortex-A76 a 2.4 GHz

80 GB RAM
16 GB por nodo

Coordinador	Raspberry Pi 4B · 8 GB · Slurm 23.11 (no calcula)
Red	Switch Gigabit en estrella
Almacenamiento	microSD de 64 GB por nodo · datos en /shared (NFSv4)
Energía	Fuente oficial de 27 W por nodo



Clúster RBPi5 en operación

#LaUISqueQueremos

Prueba inicial de potencia: HPL

Prueba estándar de supercomputación (TOP500 y Green500): mide cuántas operaciones por segundo hace cada equipo

134.86
GFLOPS
Clúster RBPi5 (5 nodos)

177.36
GFLOPS
x86 nativo

129.07
GFLOPS
WSL2

El clúster queda entre las dos laptops, con un consumo parecido (≈ 40 W)

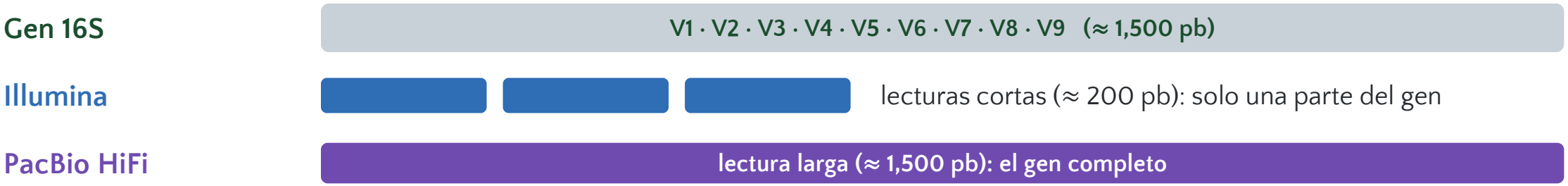
PASSED: todas las corridas pasaron la verificación

#LaUISqueQueremos

Resultados: HPL por nodo

Equipo	GFLOPS	% pico	Reposo (W)	Carga (W)	GFLOPS/W bruto	GFLOPS/W neto
rbp5-1	30.92	40.3	2.36	8.78	3.51	4.80
rbp5-2	23.23	30.2	2.39	7.86	2.95	4.25
rbp5-3	21.03	27.4	2.38	7.27	2.89	4.30
rbp5-4	28.71	37.4	2.37	8.39	3.41	4.75
rbp5-5	30.97	40.3	2.30	8.61	3.59	4.90
Clúster RBPi5 (suma)	134.86	35.1	11.80	40.90	3.25	4.58
x86 nativo	177.36	61.6	3.10	43.21	4.08	4.40
WSL2	129.07	44.8	13.05	39.47	3.23	4.82

Datasets de LEMMI16s



Kraken 2 + Bracken

Microbioma oral humano

HOMD_v4_GTDB

Illumina MiSeq · pareadas de 200 pb

2,453,431

lecturas en 5 muestras

LotuS3

Suelo contaminado con metales pesados

HM_Contaminated_Soil

PacBio HiFi · lecturas de ≈ 1,500 pb

25,611

lecturas en 5 muestras

QIIME 2

Regiones V1–V2 del gen 16S

alfa_v1v2_SILVA

Illumina · lecturas pareadas

2,690,077

lecturas en 5 muestras

Composición conocida (Ground Truth) · 3 muestras de calibración y 2 de evaluación · **mismas entradas en las 3 plataformas (MD5)**

#LaUISqueQueremos

Cómo clasifica cada pipeline

Illumina · lecturas cortas

Kraken 2 + Bracken

Sin agrupar

cada lectura por separado

- Compara cada lectura contra GTDB (k-meros)
- Bracken estima las abundancias

PacBio HiFi · gen completo

LotuS3

OTUs

grupos de lecturas parecidas

- Filtra por calidad y agrupa en OTUs
- Asigna la taxonomía con VSEARCH contra SILVA

Illumina · lecturas cortas

QIIME 2

ASVs

variantes exactas de secuencia

- Filtra por calidad y obtiene ASVs con Deblur
- Clasifica con Naive Bayes sobre SILVA V1–V2

Los tres responden lo mismo: qué bacterias hay en cada muestra y en qué proporción

#LaUISqueQueremos

Fase 2: portabilidad a ARM64

Obstáculos	Soluciones
Dificultad a la hora de utilizar contenedores provistos por LEMMI16s	**Entornos Conda nativos para aarch64**
LAMBDA3 no existe para ARM64	**usearch12 compilado. LotuS3 con VSEARCH**
QIIME2 completa no cabe en 16 GB de RAM	**SILVA recortada a V1–V2 (≈ 62,000 secuencias)**
Versiones distintas entre x86 y ARM64	**Mismas versiones en las 3 plataformas**

#LaUISqueQueremos

Resultados: fases 1 y 2

Fase 1: clúster operativo

```
grupo1@rbpi4-master:~$ sinfo -p partition1
PARTITION  AVAIL  TIMELIMIT  NODES  STATE NODELIST
partition1    up   infinite      5   idle rbp5-[1-5]
```

Los 5 nodos de cómputo disponibles (idle) en Slurm

2.38 W
por nodo en reposo

11.90 W
los 5 nodos en reposo

2.83 – 10.88 W
x86 nativo en reposo

12.95 – 15.20 W
WSL2 en reposo

Fase 2: pipelines en ARM64

```
Nodo: rbp5-1   Arquitectura: aarch64
Kraken version 2.1.3
Bracken v3.0.1
LotuS 3.03
q2cli version 2022.8.0
grupo1@rbpi4-master:~$
```

Los 3 pipelines corren nativos en aarch64

Mismas versiones que en x86

#LaUISqueQueremos

Fases 3 y 4: validación y medición

Validación (Fase 3)

Mismos datos
5 muestras por dataset, idénticas en las 3 plataformas (MD5)

F1 por muestra
precisión y sensibilidad frente al Ground Truth

Salidas comparadas
archivos de las 3 plataformas, línea por línea (diff -u)

60 s
estabilización

→

5 min
reposo (P idle)

→

Pipeline
medido a 1 Hz

Medición de energía (Fase 4)

Potencia a 1 Hz, integrada por la regla del trapecio (Wh)

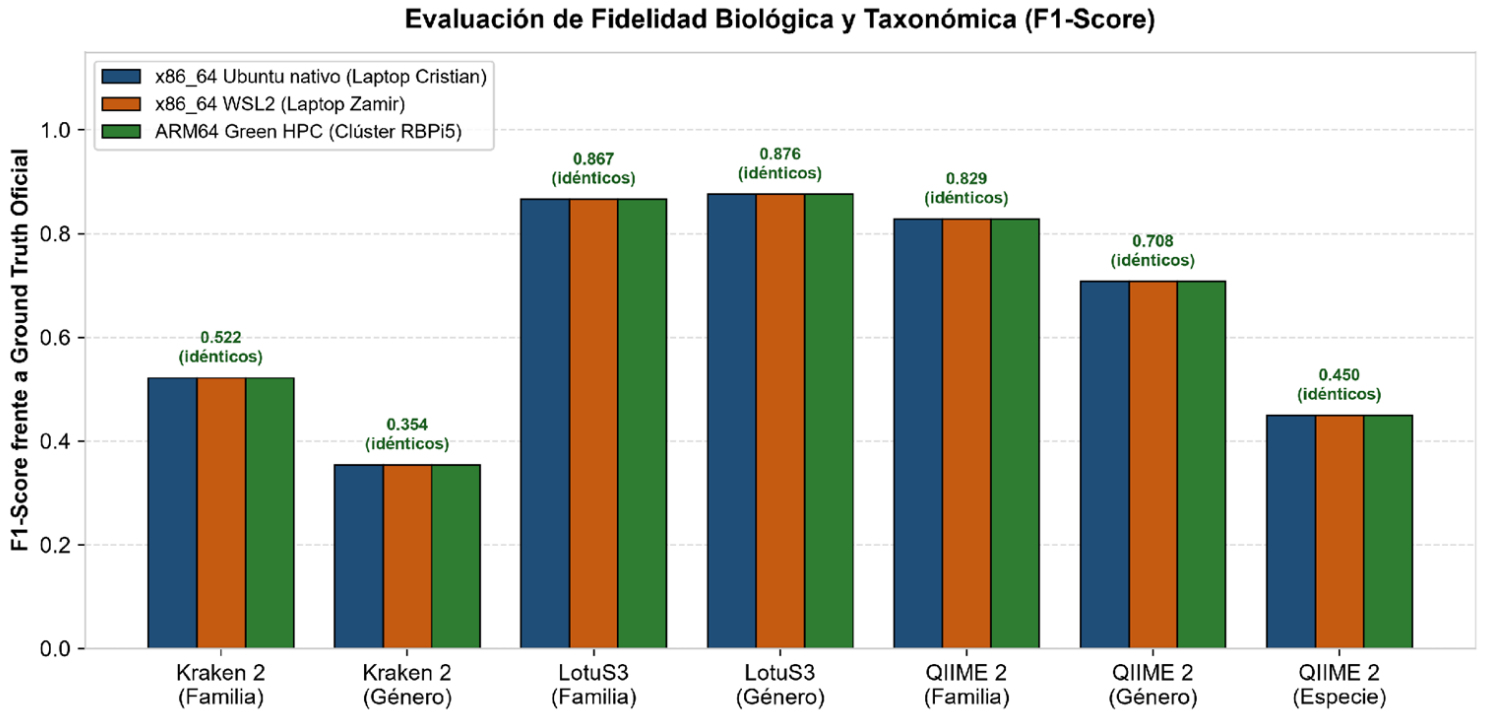
$$E_{total} = \Sigma \tfrac{1}{2} (P_i + P_{i+1}) \cdot \Delta t$$
toda la energía medida durante el pipeline

$$E_{neta} = E_{total} - P_{idle} \cdot t$$
lo que costó el cálculo, sin el reposo

$$\eta = \text{lecturas} / Wh$$
eficiencia energética

#LaUISqueQueremos

Resultados Fase 3: fidelidad biológica



Consistencia de las salidas

- **Kraken 2 y QIIME 2:** tablas idénticas bit a bit
- **LotuS3:** mismas familias y géneros (pequeñas variaciones en abundancias)

Mismo F1-Score

Resultado robusto e independiente de la plataforma

#LaUISqueQueremos

Resultados Fase 4: Tiempo en 1 nodo frente a 5 nodos

4.47× · 4.52× · 1.94×

1 nodo, más lento que x86 nativo (Kraken 2 · LotuS3 · QIIME 2)

2.58×

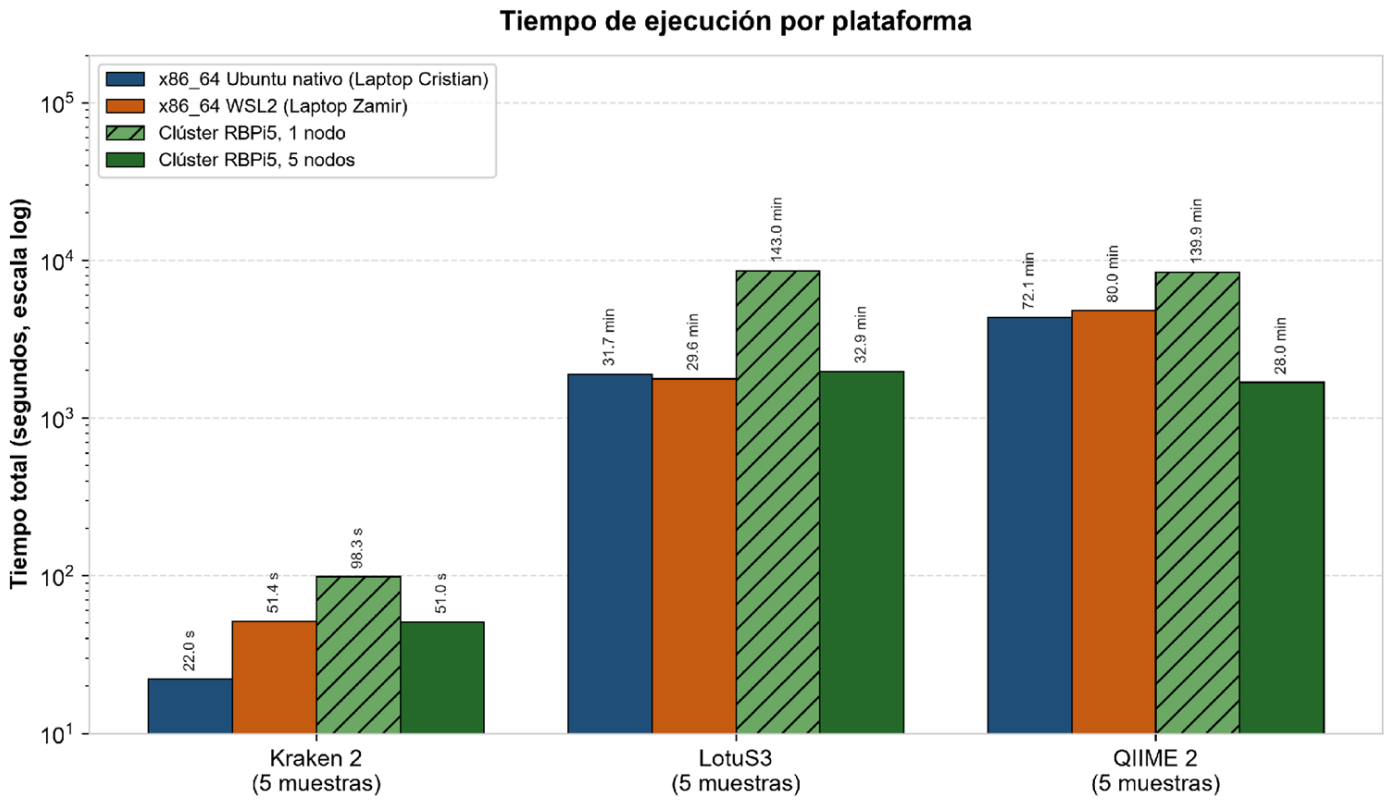
QIIME 2 en 5 nodos, más rápido que x86 nativo

0.96×

LotuS3: 32.87 frente a 31.65 min

2.32×

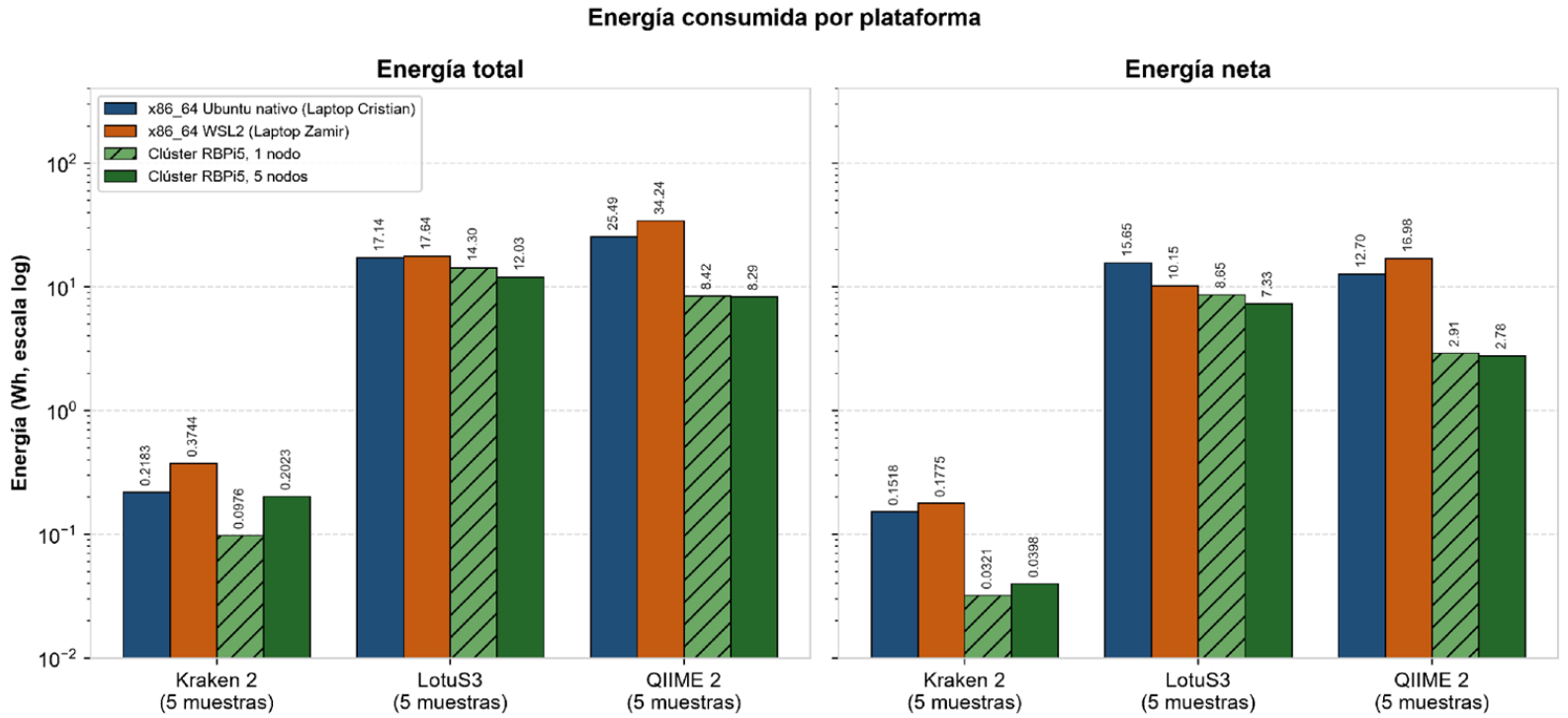
Kraken 2: x86 nativo más rápido (NFS)



Tiempo de ejecución (escala log)

#LaUISqueQueremos

Energía en 1 nodo frente a 5 nodos



4.7× · 1.8× · 4.4×
menos energía neta con 1 nodo que x86 nativo

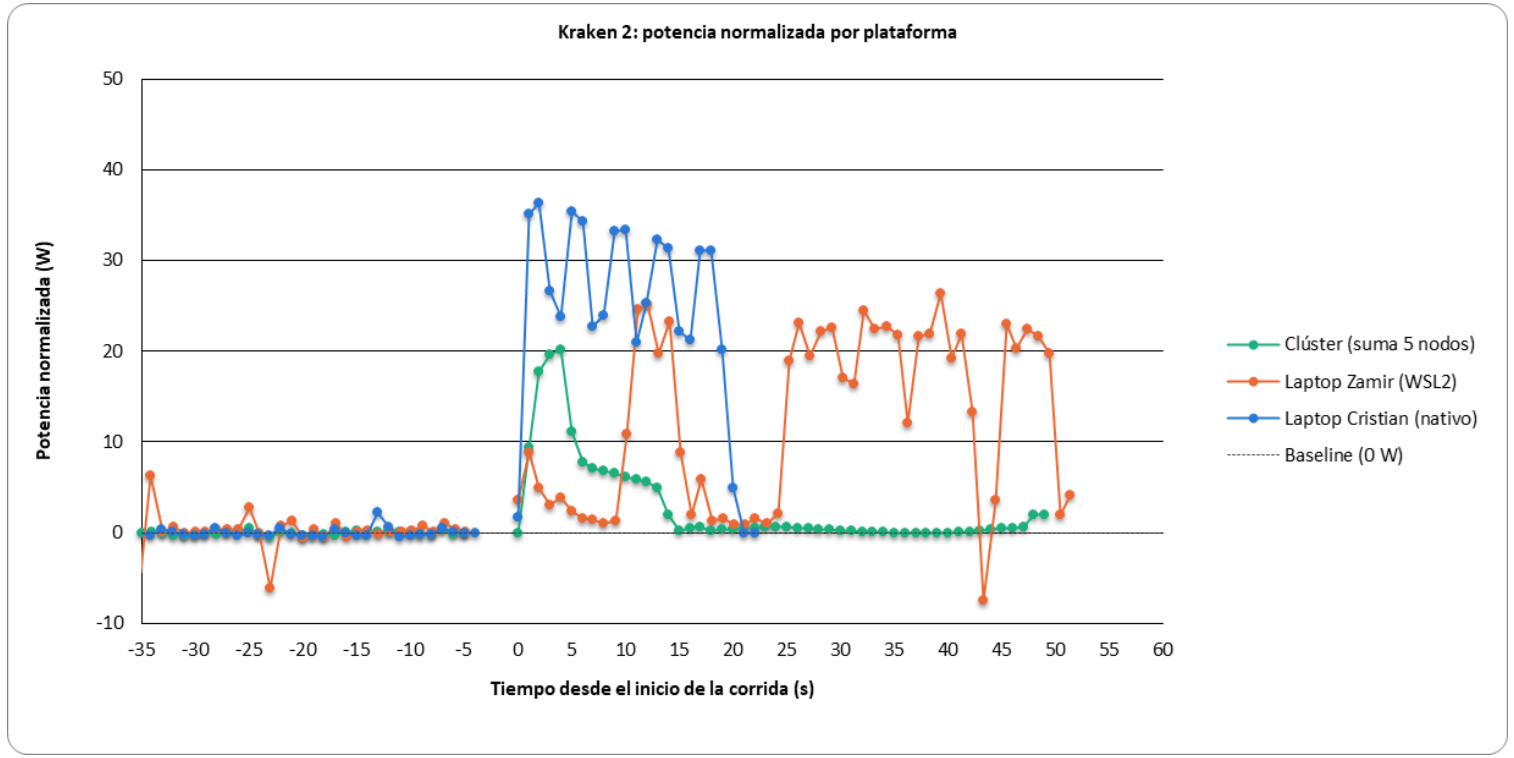
8.42 → 8.29 Wh
QIIME 2: total con 1 y con 5 nodos

14.30 → 12.03 Wh
LotuS3: total con 1 y con 5 nodos

0.0976 → 0.2023 Wh
Kraken 2: se duplica por la espera en NFS

#LaUISqueQueremos

Kraken 2 + Bracken



22.01 s · 51.04 s

x86 nativo · clúster

3.81×

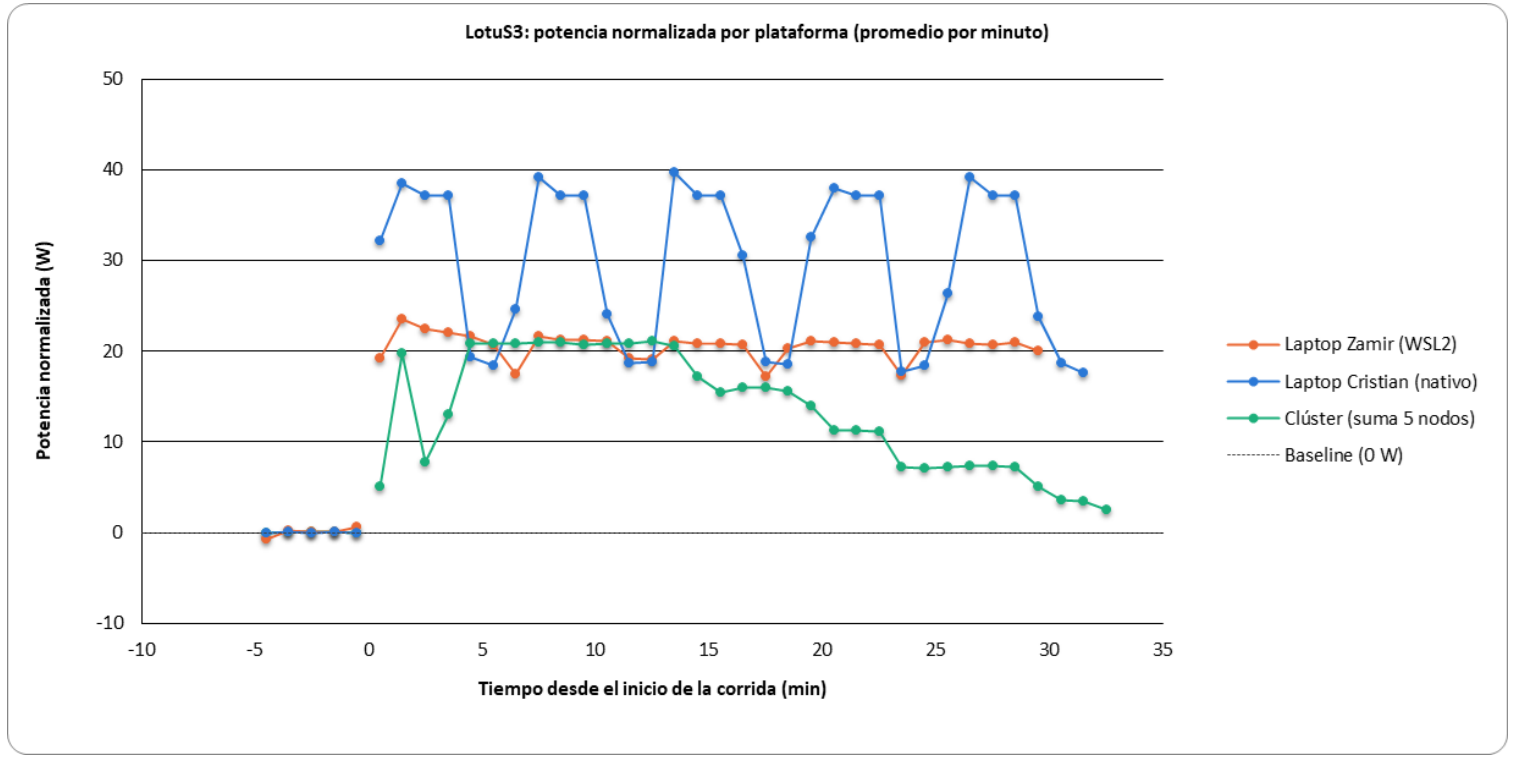
menos energía neta (0.0398 frente a 0.1518 Wh)

1.08×

en energía total: domina el reposo

#LaUISqueQueremos

LotuS3

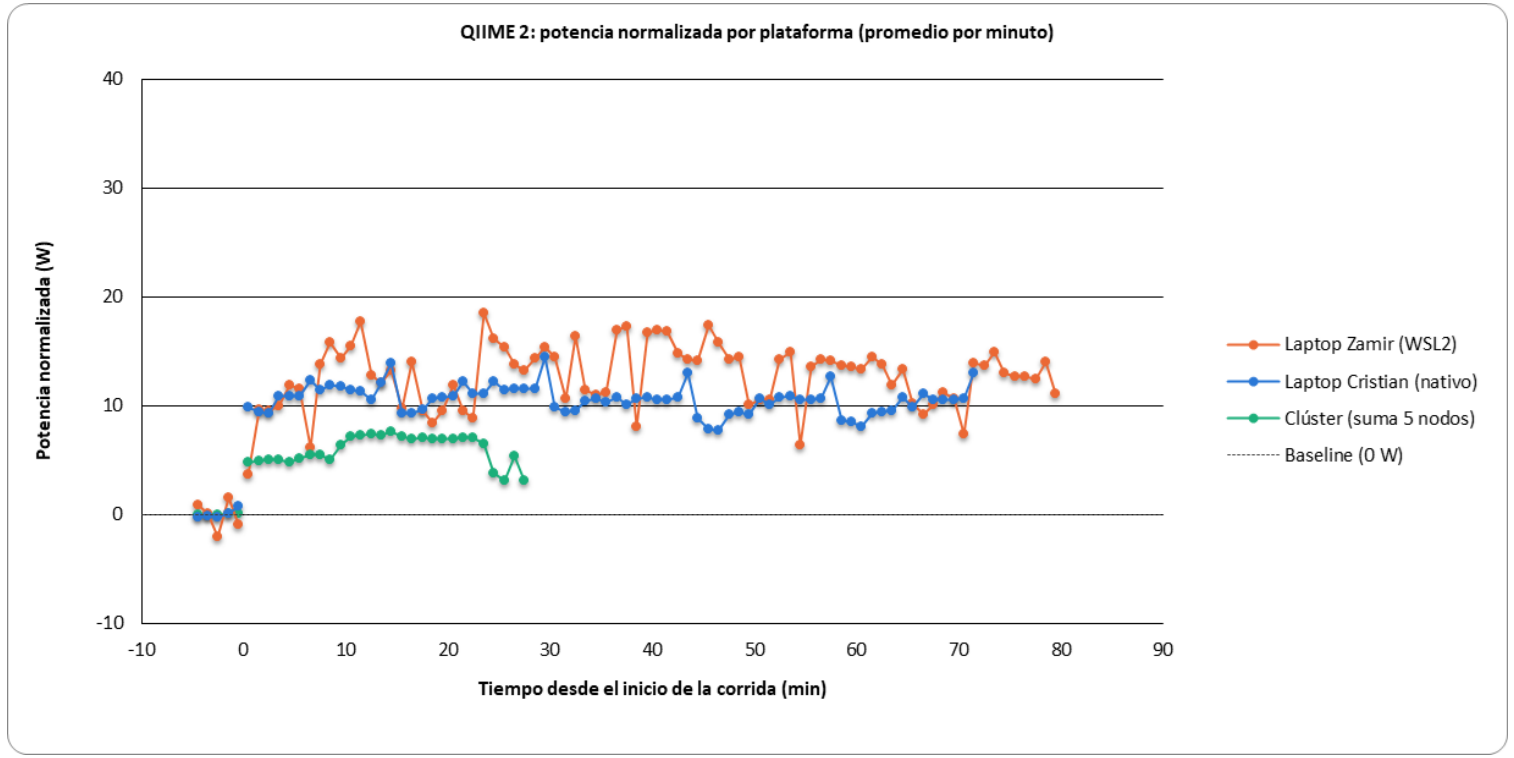


32.87 min
clúster (nativo 31.65 · WSL2 29.60)

14 a 33 min
según el nodo: espera de datos en NFS

2.14×
menos energía neta (7.33 frente a 15.65 Wh)

QIIME 2



27.98 min
frente a 72.06 en x86 nativo (2.58×)

5.00×
aceleración de 1 a 5 nodos

2.78 Wh
energía neta frente a 12.70 (más de 78 % menos)

Eficiencia energética (reads/Wh)

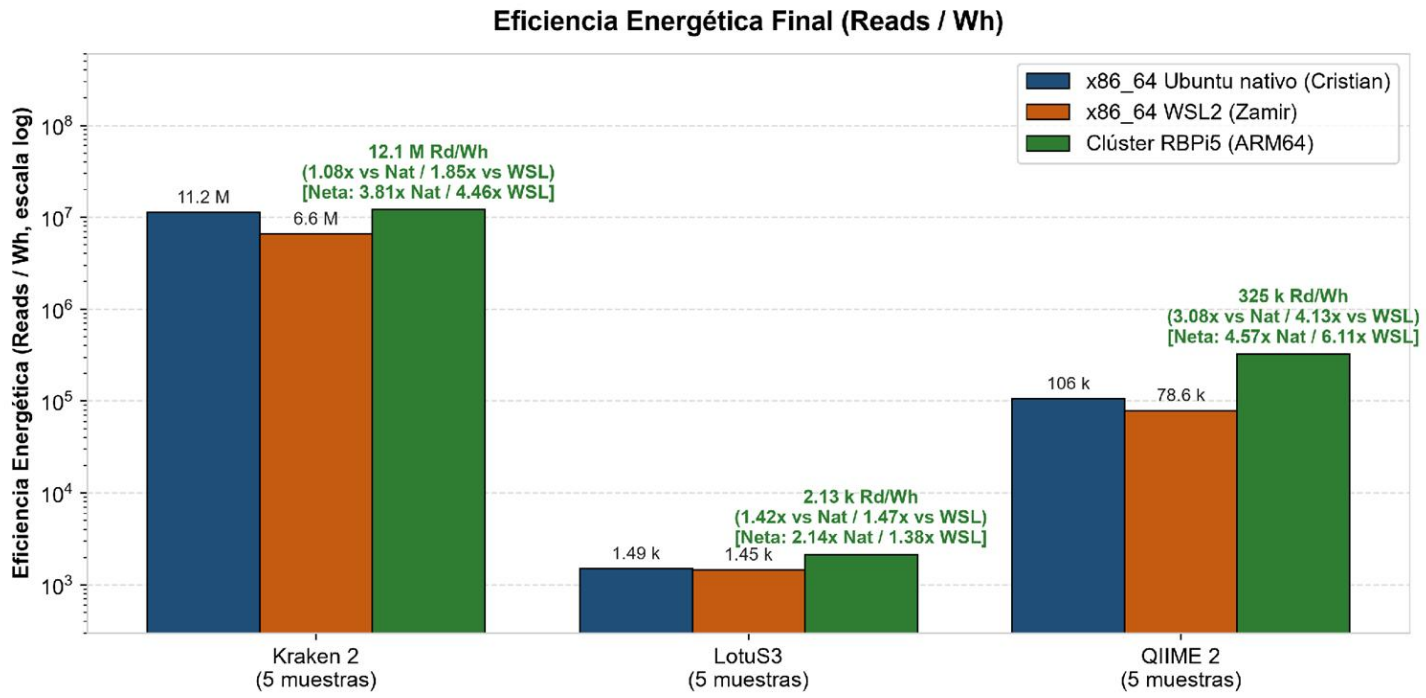


Figura 3.19 del libro (escala log, energía total)

Kraken 2

12.1 M lecturas/Wh

x86 nativo: 1.08× bruta · **3.81× neta**

LotuS3

2,129 lecturas/Wh

x86 nativo: 1.42× bruta · **2.14× neta**

QIIME 2

324,561 lecturas/Wh

x86 nativo: 3.08× bruta · **4.57× neta**

PMIC ≠ RAPL

factores observados (placa frente a procesador)

#LaUISqueQueremos

Conclusiones (1/2)

Fase 1 · Diseñar ✓

2.38 W

por nodo en reposo

Clúster de 5 nodos operativo con Slurm y NFS

Fase 2 · Portar ✓

3 de 3

pipelines nativos en ARM64

Sin contenedores y con las mismas versiones que x86

Fase 3 · Validar ✓

F1 idéntico

en las 3 plataformas

Misma taxonomía que en x86

Objetivos 1, 2 y 3 cumplidos: clúster operativo, pipelines portados y resultados validados; la base para medir tiempo y energía en la Fase 4

#LaUISqueQueremos

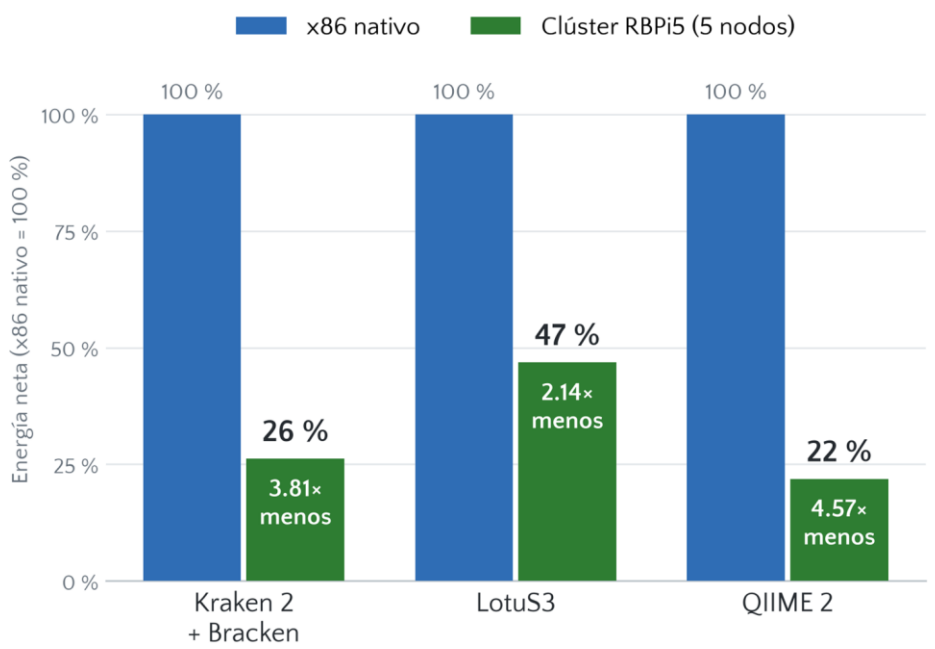
Conclusiones (2/2)

Fase 4

Menos energía neta en los 3 pipelines

Más rápido al repartir: **5.00× QIIME 2** (1 nodo vs 5 nodos)

La **red y el NFS** limitan Kraken 2 y LotuS3



Respuesta a la pregunta: el clúster logra hasta 3,08× más lecturas por Wh que x86 en energía total, con la misma taxonomía y mejores tiempos al distribuir el procesamiento sin depender de almacenamiento compartido.

#LaUISqueQueremos

Limitaciones y trabajo futuro

Limitaciones

- Datos sintéticos de LEMMI16s
- LotuS3 con VSEARCH (sin LAMBDA3)
- SILVA recortada a V1–V2 en QIIME 2
- Red Gigabit y NFS compartidos
- PMIC (placa) frente a RAPL (procesador)

Trabajo futuro

10 nodos + 1 disco duro (PCIe a M.2)

Quitar el cuello de botella del almacenamiento compartido

DADA2 en ARM64

Portar este otro método de ASVs, escrito en R

SILVA completa

Clasificar sin recortar la base: más memoria o intercambio en NVMe

Energía solar (MPPT)

Cargar baterías y usar el clúster sin red eléctrica

#LaUISqueQueremos

Sustentación de trabajo de grado

Cristian Alberto Solano Torres
Zamir Francisco Granados Peñaloza

Director: Bernardo Andrés Benavides Arévalo, PhD

github.com/CristianSolanoUIS/lemmi16s-cluster-RbPi5

¡Gracias!

Preguntas y discusión con el jurado